

A Fast Algorithm for Generating Fuzzy Rules Online by Incremental Mining Approach

Ming-Yang Su

Department of Computer Science and Information Engineering,
Ming Chuan University, Taoyuan Campus, Taiwan
minysu@mcu.edu.tw

Abstract

In the paper, we propose a fast fuzzy association rules generating algorithm that can do once mining from milliseconds to seconds, by incremental mining. Extensively experiments and analyses are done to show the performance of the algorithm, especially for costs of memory and time consumption.

1. Introduction

Apriori's algorithm [1] is the most famous and basic method to derive a set of association rules from a static database. Since it is designed for static database with Boolean attributes and very time consuming, many variations have been proposed to upgrade the algorithm in the literature. Incremental mining [2, 3] and fuzzy mining [5, 6] are two branches of these variations. Recently, on-line mining [7, 8] has got a lot of attention because many applications need to get response as soon as it could be. Mining time is critical for some applications, especially for Network Intrusion Detection Systems (NIDSs).

Several NIDSs [4, 9] adopted fuzzy association rule to analyze network traffic, but none of them can really achieve so called on-line mining. This is because they are basically

designed for static mining, i.e., they have to collect packet information for a fairly long period of time and then analyze the information later. If the time for collecting traffic information is too short, the performance (detection rate, false alarm rate, etc) would not be good. However, if the time for collecting traffic information is too long, the NIDS can not react immediately and thus the mining result become useless.

Theoretically, incremental fuzzy mining can get the same rules as static fuzzy mining on the same datasets. In addition, incremental fuzzy mining is much suitable to meet real-time demand because it can produce the latest set of rules while a new record of packet information is gathered. In this paper we present an on-line incremental mining algorithm for generating fuzzy association rules. The algorithm is fast enough for most NIDS designs.

The rest of this paper is organized as follows. Some background knowledge is introduced in Section 2. Our algorithm is described clearly in Section 3. Its performance analysis and experimental results are shown in Section 4. Finally, conclusion is given in Section 5.

2. Background

We briefly introduce association rules and fuzzy association rules in Subsections 2.1 and 2.2, respectively.

2.1 Association Rules

An association rule has the form of "If X , then Y ", denoted as $X \Rightarrow Y$, where X and Y are sets of items (or called itemsets). The formal statement about association rules is given as: Let $I = \{i_1, i_2, i_3, \dots, i_m\}$ be a set of items. Let D be a set of transactions, where each transaction T is a set of items such that $T \subseteq I$. We say that a transaction T contains an itemset X if $X \subseteq T$. An *association rule* is an implication of the form $X \Rightarrow Y$, where $X \subset I$, $Y \subset I$, and $X \cap Y = \emptyset$. The rule $X \Rightarrow Y$ in the transaction set D has *support* s if $s\%$ of transactions in D contain $X \cup Y$, and has *confidence* c if $c\%$ of transactions in D that contain X also contain Y . That is, let $\text{Sup}(X)$ denote the occurrence frequency of the itemset X in D , $s = \text{Sup}(X \cup Y)$ and $c = \text{Sup}(X \cup Y) / \text{Sup}(X)$.

Agrawal and Srikant proposed the well known Apriori algorithm [1] in 1994, in which given two thresholds of *mini_sup* and *mini_conf*, the algorithm will find all such rules as $X \Rightarrow Y$ with $s \geq \text{mini_sup}$ and $c \geq \text{mini_conf}$. For any itemset, say X , with $\text{Sup}(X)$ greater than or equal to the *mini_sup*, is called *frequent* itemset. Let k -itemset denote an itemset having k items. Let L_k represent the set of frequent k -itemsets, and C_k the set of all possible (or candidate) k -itemsets. The first phase of Apriori algorithm is to generate all frequent itemsets described as follows:

Step 0. Scan the database D to find out L_1 .

Step 1. C_k , $k > 1$, is generated by joining the itemsets in L_{k-1} . The itemsets in C_k which have some $(k-1)$ -subset that is not in L_{k-1} are deleted.

Step 3. The support of itemsets in C_k is calculated through database scan to decide L_k .

Step 4. Go to Step 2 until $L_k = \emptyset$.

The main concept behind the algorithm is that all subsets of a frequent itemset must be frequent. After the first phase, Apriori algorithm's second phase is to compute supports of all nonempty subsets of each frequent itemset, in order to derive qualified rules which confidences are greater than or equal to the *mini_conf*. Association rules mining has become a focal point of research and application in data mining since Apriori algorithm was proposed.

2.2 Fuzzy Association Rules

Apparently Apriori algorithm was designed for mining in databases with binary features (item and feature being used interchangeably throughout the paper). Fuzzy association rules mining [5, 6] was one of those variations to deal with quantitative features. Contrast to classical approach like partitioning feature domains, fuzzy logic can solve the sharp boundary problem [10]. The definition of fuzzy association rule is stated as follows. Let $T = \{t_1, t_2, \dots, t_n\}$ be the database and each transaction t_i represents the i th tuple in T . Moreover, we use $I = \{i_1, i_2, \dots, i_m\}$ to represent all features appeared in T . Each quantitative feature i_k , $1 \leq k \leq m$, is associate with some fuzzy

variables, say $v_1, v_2, \dots, v_j$. Every fuzzy variable is represented by a membership function. For easy representation in the following, we use $F_{i_k.v_j}$ to uniquely denote the j th membership function of feature i_k .

Suppose in some network traffic analyzing application, four features, say *#packet*, *#SYN*, *#ACK*, and *#connection*, are concerned and each feature has three fuzzy variables, say *low*, *medium*, and *high*, then $4 \times 3 = 12$ membership functions are involved. $F_{\#packet.low}$ denote the low function of feature *#packet*, $F_{\#ACK.high}$ denotes the high function of feature *#ACK*, etc. The number of fuzzy variables for each feature and their corresponding membership functions are provided by domain experts.

In fuzzy association rule mining, an fuzzy itemset consists of two parts: items and fuzzy variables, say $\langle X, V \rangle$, where $X = (x_1, x_2, \dots, x_i) \subset I$ is a collection of items (or features) and $V = (v_1, v_2, \dots, v_i)$, is the collection of corresponding fuzzy variables to X in order. For instance, if $X = (\#packet, \#SYN, \#ACK, \#connection)$ and $V = (low, medium, low, high)$, the fuzzy itemset $\langle X, V \rangle$ represents $\{\#packet \text{ is } low, \#SYN \text{ is } medium, \#ACK \text{ is } low, \#connection \text{ is } high\}$.

The support of $\langle X, V \rangle$ is computed as [6]:

$$\text{Sup}(\langle X, V \rangle) \quad (1)$$

$$= \frac{\text{Sum of votes satisfying } \langle X, V \rangle}{\text{number of records in } T}$$

$$= \frac{\sum_{t_i \in T} \prod_{x_j \in X} F_{x_j.s_j}(t_i[x_j])}{|T|},$$

where $t_i[x_j]$ denotes the value of feature x_j in the i th record.

Instead of "If X , then Y " in association rule, a fuzzy association rule now has the form of "If $\langle X, V \rangle$, then $\langle X', V' \rangle$ ". Here $\langle X, V \rangle$ and $\langle X', V' \rangle$ are two itemsets, $X \subset I$, $X' \subset I$, and $X \cap X' = \emptyset$. The first part $\langle X, V \rangle$ is called antecedent and the second part $\langle X', V' \rangle$ is called consequent. Similarly support s and confidence c of the fuzzy association rule $\langle X, V \rangle \Rightarrow \langle X', V' \rangle$ are computed as

$$s = \text{Sup}(\langle Y, U \rangle) \text{ and}$$

$$c = \text{Sup}(\langle Y, U \rangle) / \text{Sup}(\langle X, V \rangle),$$

where $\langle Y, U \rangle$ being the concatenation itemset of $\langle X, V \rangle$ and $\langle X', V' \rangle$. For example, if $X = (x_1, x_2, \dots, x_i)$, $V = (v_1, v_2, \dots, v_i)$, $X' = (x'_1, x'_2, \dots, x'_j)$, and

$$V' = (v'_1, v'_2, \dots, v'_j), \text{ then } Y = (x_1, x_2, \dots, x_i, x'_1, x'_2, \dots, x'_j) \text{ and } U = (v_1, v_2, \dots, v_i, v'_1, v'_2, \dots, v'_j).$$

The fuzzy association rule has received a great attention recently and has been applied to many applications in different fields [11, 12]. Several researchers adopted fuzzy association rule to design their NIDSs [4, 9]. However, they [4, 9] all collected records of packet information, and mined these records later in a static way.

3. Our Incremental Mining Algorithm for Fuzzy Association Rules

While adopting fuzzy association rules to analyze network traffic in real-time especially for

the designs of NIDS, the time cost of mining become vital. On the other hand, if not enough packets are observed, the detection performance will not be good because some attack events may last for a long period of time. Our incremental mining approach can meet the two requirements. The basic concept of our algorithm is given in Subsection 3.1. Detailed implementation is followed in Subsection 3.2.

3.1 Incremental Mining for Fuzzy Association Rules

In order to meet the real-time requirement, we use an incremental mining way to derive the fuzzy association rule. An example is illustrated to show the computation of equation (1) by incremental mining. Suppose the quantitative values of features of *#packet*, *#SYN*, *#ACK*, and *#connection*, are measured for each time slot, and in sequence they are $t_1 = (70, 6, 5, 12)$, $t_2 = (59, 8, 4, 3)$, $t_3 = (90, 12, 6, 9)$, ..., $t_i = (a, b, c, d)$. For the fuzzy itemset $\langle X, V \rangle = \{\#packet \text{ is } low, \#SYN \text{ is } medium, \#connection \text{ is } low\}$, its support value s in $t_1, t_2, t_3, \dots, t_i$ is computed as :

$$\begin{aligned}
 s \text{ in } t_1 &= [F_{\#packet,low}(70) \times F_{\#SYN,medium}(6) \times \\
 &\quad F_{\#connection,low}(12)] / 1 ; \\
 s \text{ in } t_2 &= \{ s \text{ in } t_1 \times 1 + [F_{\#packet,low}(59) \times \\
 &\quad F_{\#SYN,medium}(8) \times F_{\#connection,low}(3)] \} / 2 ; \\
 s \text{ in } t_3 &= \{ s \text{ in } t_2 \times 2 + [F_{\#packet,low}(90) \times \\
 &\quad F_{\#SYN,medium}(12) \times F_{\#connection,low}(9)] \} / 3 ; \\
 &\dots \\
 s \text{ in } t_i &= \{ s \text{ in } t_{i-1} \times (i - 1) + [F_{\#packet,low}(a) \times \\
 &\quad F_{\#SYN,medium}(b) \times F_{\#connection,low}(d)] \} / i.
 \end{aligned}$$

Since contribution of previous records to the current support can be ephemerally saved to a temporary variable, the cost of mining time wouldn't be prolonged as observed records increase. Time costs for static mining and incremental mining are illustrated in Figure 1, whereas both get the same rule set.

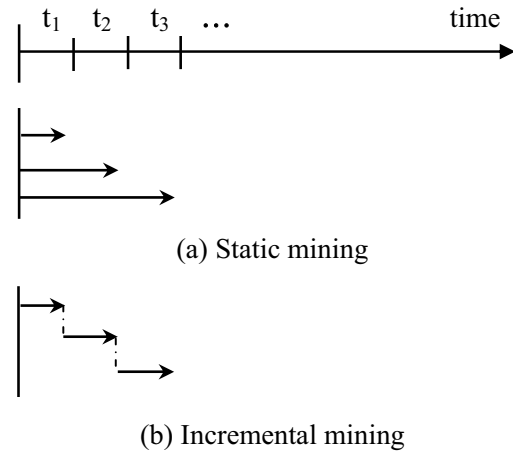


Figure 1. Time costs for static mining and incremental mining

3.2 Detailed Description of Our Algorithm

Based on the above example, we perceive that all possible fuzzy itemsets should be stored in the main memory to speed up mining processes. Moreover, each itemset must be associated with a variable to store its current support value. Since the number of possible itemsets could be very large, the difficulty firstly confronted is how to represent, generate and organize such tremendous itemsets in the main memory.

3.2.1 Data Structure

We use an alphabet character to denote one feature's fuzzy variable in our algorithm. So if there are n features and each has m fuzzy variables, then starting at character 'A', $n \times m$ contiguous characters are used. These characters can be regarded as placed on an $n \times m$ matrix in row major. For the example of $n = 6$ and $m = 3$, the matrix is shown in Figure 2. The benefit of such representation is that for any character, it has three notations, i.e., ASCII value, index position, and fuzzy variable, and translations among these notations are very simple. Let us take the character 'G' having ASCII value 71 for explanation. Since the ASCII value of 'A' is 65, G's index position denoted by $(row, column)$ can be computed as $row = \text{integer}[(71 - 65) / 3] = 2$ and $column = (71 - 65) \bmod 3 = 0$. Based on the index position (2, 0), the character 'G' also denotes the third feature's first fuzzy variable, i.e., $F_{f3,low}$.

		low	medium	high
		0	1	2
f1	0	A	B	C
f2	1	D	E	F
f3	2	G	H	I
f4	3	J	K	L
f5	4	M	N	O
f6	5	P	Q	R

Figure 2. Relations with character, index position, and fuzzy variable

In case a feature doesn't possess any one of these three fuzzy variable, simply replace the corresponding position with a specified character,

such as asterisk (*). With slightly modification, the algorithm presented in the section is still work.

With such arrangement each fuzzy itemset can be expressed by a string to reduce memory consumption. For instance, string "EJR" denotes the fuzzy itemset {f2 is medium, f4 is low, f6 is high}. Since each itemset is associated with a support value, we declare a node structure as follows. In our algorithm, each fuzzy itemset is represented by a node.

```
struct itemset {
    float SupVal;
    char STRING[maxlen];
    struct itemset *next;
};
```

The *maxlen* is defined as the longest length of itemset, which theoretically is less than or equal to the number of features. Fortunately, *maxlen* in practical is much less than the number of features; this claim is affirmed in Section 4.

Our algorithm consists of three procedures: *Initialization*, *SupCal*, and *RuleGen*. All nodes are constructed and linked by the *Initialization*. Support value of each node is frequently updated by the *SupCal*; meanwhile for any node (i.e., fuzzy itemset) with support value greater than or equal to the *mini_sup*, the *RuleGen* is called to test all possible rules and print out those rules with confidences greater than or equal to the *mini_conf*.

3.2.2 Initialization procedure

In the initialization, we construct nodes to keep all possible itemsets in the memory and use linked lists to link nodes by their lengths. Again the example in Figure 2 is used for easy illustration. All possible fuzzy itemsets are linked

as shown in Figure 3. The last node of each list is pointed to the NULL.

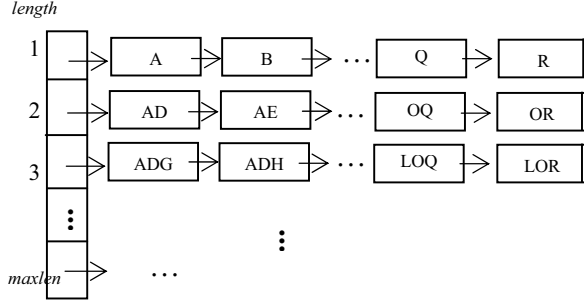


Figure 3. All possible fuzzy itemsets

This work can be done by the following **Initialization** procedure. Let n be the number of features and m be the number of degrees (fuzzy variables). The procedure maintains an $n \times m$ matrix $Matrix[n][m]$, using the number of '1' and their positions in the matrix to represent what node currently being created. Also let $maxlen \leq n$ denote the longest length of fuzzy itemset. If we set $maxlen$ to k , say 2, all itemset with length greater than k , i.e., 2, are ignored.

Procedure Initialization

//Input: features n , degrees m , and longest itemset length $maxlen \leq n$

//Output: $maxlen$ linked lists in memory; for each list, nodes have the same length and appear in lexicographic order.

//Assumption: $Matrix[][]$ and $length$ are two global variables.

```

01  $Matrix[i][j] \leftarrow 0$ , for all  $i$  and  $j$ ;    //zero matrix
02 for ( $length = 1$ ;  $length \leq maxlen$ ;  $length++$ ) {
     $CreateLinkedList(length, 0)$ ;
03 }
```

SubProcedure CreateLinkedList (int len, int start)

```

01 if  $len = 1$ , then {
02     for ( $i = start$ ;  $i < n$ ;  $i++$ ) {
03         for ( $j = 0$ ;  $j < m$ ;  $j++$ ) {
04              $Matrix[i][j] \leftarrow 1$ ;
05             Scan  $Matrix$  from top to down and left to
                right to find out all positions set to 1;
06             Dynamically allocate a node and use the
                 $STRING$  field to store the
                corresponding fuzzy itemset;
07             Add the node to the tail of the list with
                length  $length$ ;
08              $Matrix[i][j] \leftarrow 0$ ;
09         }
10     }
11 } //end if
12 else {    //  $len > 1$ 
13     for ( $i = start$ ;  $i \leq n - len$ ;  $i++$ ) {
14         for ( $j = 0$ ;  $j < m$ ;  $j++$ ) {
15              $Matrix[i][j] \leftarrow 1$ ;
16              $CreateLinkedList(len - 1, i + 1)$ ;
17              $Matrix[i][j] \leftarrow 0$ ;
18         }
19     }
20 } //end else
```

Clearly, by the for-loop in the Initialization procedure linked lists can be constructed from length 1 to $maxlen$. Following the above example in Figure 2 and Figure 3, let's see how to construct the list of length 3 for instance, i.e., the construction of $CreateLinkedList(3, 0)$. At the first, $Matrix$ is set to a zero matrix. Since now length equal to 3 is considered, the recursive subprocedure $CreateLinkedList$ has to exhaustively display all possible arrangements in $Matrix$ satisfied with only three '1's are appeared and no any two of them are located in the same row. Each possible arrangement is replaced by a node, and $STRING$ field of the node stored the

corresponding characters in *Matrix* which positions are marked '1'. Two feasible arrangements are shown in Figure 4. The scan of matrix is from top to down and left to right, so that characters in the *STRING* field of each node are in lexicographic (or ASCII) order. This property is essential for the *RuleGen* procedure presented in Subsection 3.2.4.

1	0	0
0	0	0
0	0	1
0	1	0
0	0	0
0	0	0

"AIK"

0	0	0
0	1	0
0	0	0
0	0	0
1	0	0
0	0	1

"EMR"

Figure 4. Two feasible fuzzy itemsets

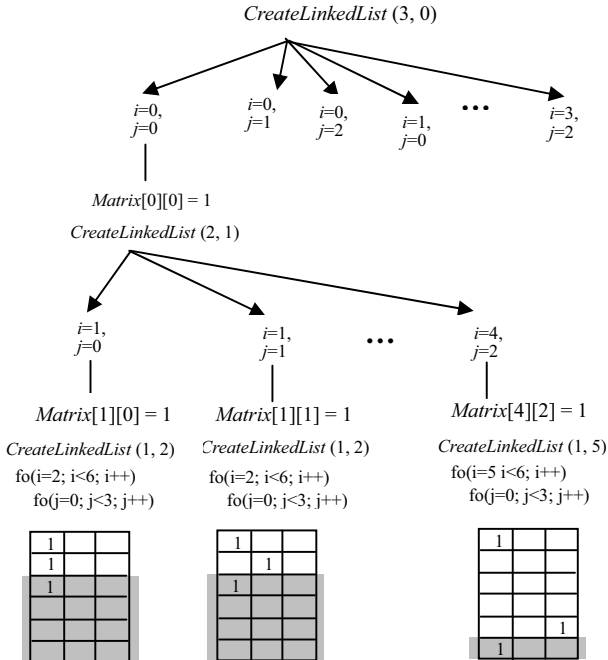


Figure 5. The construction of *CreateLinkedList*(3, 0)

The construction of *CreateLinkedList*(3, 0) is illustrated in Figure 5. In this figure, the last, i.e. third, '1' in the shaded area is rolling in row major on every position in the area by nested for-loop (lines 2 and 3 in the subprocedure).

3.2.3 SupCal procedure

3.2.4 RuleGen procedure

Both of the above two procedure are omitted here; details can be found in [13].

4. Performance Analysis

Extensively performance analysis is discussed in the section; especially the memory consumption and time cost are most concerned. Our program is coded by C++ and run on Windows XP platform with P4-3.2GHZ CPU and 1024Mbytes memory. Fuzzy membership functions applied list below. The output value of any membership function is falling in the interval [0, 1]. Each increasing record is randomly generated with features values ranging in the interval [1, 500].

$$\text{Low} : f(x) = 1 / (1 + \exp(0.05(x - 130)))$$

$$\text{Medium} : f(x) = 1 / (1 + |(x - 250) / 80|^4)$$

$$\text{High} : f(x) = 1 / (1 + \exp(-0.05(x - 380)))$$

According to our experiment, the longest itemset length (*maxlen*) that our algorithm concerned set to 6 is enough. Let *n* be the number of features, *m* be the number of degrees (fuzzy

sets) for each feature, and k , $1 \leq k \leq n$, be the longest itemset length. As $m = 3$, $k = 8$, $mini_sup = 0.2$ and $mini_conf = 0.4$, the number of rules generated from different lengths of itemsets are shown in Table 1. All of rules were derived from itemsets of lengths less than or equal to 6. In the experiment, two hundreds records are randomly generated; once mining is done after one record is randomly generated. So two hundreds of mining were done, the numbers of rules in total by their lengths were counted. Value in parenthesis is the average.

Table 1. Numbers of rules derived from variant itemset lengths as $m = 3$, $k = 8$, $min_sup = 0.2$, $min_conf = 0.4$

rules	$n = 9$	$n = 12$	$n = 15$	$n = 18$	$n = 21$
$len = 1$	0 (0)	0 (0)	0 (0)	0 (0)	0 (0)
$len = 2$	5583 (27.91)	6017 (30.08)	6363 (31.81)	5953 (29.76)	5940 (29.70)
$len = 3$	957 (4.79)	1025 (5.13)	1361 (6.81)	945 (4.73)	1560 (7.80)
$len = 4$	1004 (5.02)	992 (4.96)	410 (2.05)	938 (4.69)	796 (3.98)
$len = 5$	708 (3.54)	663 (3.31)	74 (0.37)	696 (3.48)	255 (1.28)
$len = 6$	214 (1.07)	178 (0.89)	0 (0)	242 (1.21)	47 (0.24)
$len = 7$	0 (0)	0 (0)	0 (0)	0 (0)	0 (0)
$len = 8$	0 (0)	0 (0)	0 (0)	×	×

The definitions about n , m , and k defined above are used throughout the rest of this section. It is not difficult to compute that totally $\sum_{x=1}^k m^x C_x^n$ nodes will be constructed by our algorithm in the *Initialization* procedure, where C is the combination operation. Moreover, these nodes depending on their itemset length are

connected to k linked lists. As $m = 3$, $2 \leq k \leq 6$, and $3 \leq n \leq 30$, the number of nodes and their memory consumptions are counted in Table 2. Each node is counted as 12 bytes if $k = 2$ or 4, and 16 bytes if $k = 6$.

Table 2. Memory consumptions with variant n and

k

$n \backslash k$		#nodes	KBytes	MBytes
6	4	1908	22.35	0.022
	5	3366	52.59	0.051
	6	4095	63.98	0.062
9	4	12825	150.29	0.147
	5	43443	678.79	0.663
	6	104679	1635.61	1.597
12	4	46665	546.85	0.534
	5	239121	3736.26	3.65
	6	912717	14261.20	13.927
15	4	123840	1451.25	1.417
	5	853569	13337.01	13.02
	6	4502214	70347.09	68.698
18	4	271323	3179.56	3.105
	5	2353347	36771.04	35.91
	6	15886503	748226.61	242.409
21	4	522648	6124.78	5.981
	5	5467455	85428.98	83.43
	6	45025911	703529.86	687.041
24	4	917910	10756.76	10.505
	5	11246382	175724.71	171.61
	6	109366866	1708857.28	1668.806

The memory demand is increased fast. However, nowadays computer usually equipped with hundreds or even thousand of Mbytes. If the number of features is no more than 21, our algorithm can work fine in normal Personal Computer. Next we evaluate time cost for once mining. As one record is randomly generated, our program is run once; with two hundreds records in

total, our program is run 200 times. Fixed $m = 3$, under different conditions the time spent for once mining in average is given in Table 3.

Table 3. Time costs with variant n and k

seconds		$k = 4$	$k = 5$	$k = 6$
$n = 6$	a	0.00149	0.00258	0.00344
	b	0.00125	0.00242	0.00320
	c	0.00118	0.00250	0.00305
$n = 9$	a	0.00672	0.02532	0.06719
	b	0.00610	0.02610	0.06891
	c	0.00594	0.02586	0.06851
$n = 12$	a	0.01906	0.11828	0.49907
	b	0.01836	0.12344	0.51156
	c	0.01852	0.12289	0.51687
$n = 15$	a	0.04375	0.38110	2.22039
	b	0.04282	0.39672	2.25758
	c	0.04297	0.40110	2.26899
$n = 18$	a	0.08992	0.97766	7.39321
	b	0.08742	1.01344	7.37078
	c	0.08610	1.00469	7.35562
$n = 21$	a	0.15860	2.20594	19.59477
	b	0.15563	2.21297	19.57117
	c	0.15641	2.21422	19.51157

Note: a: $mini_sup = 0.1, mini_conf = 0.3$

b: $mini_sup = 0.3, mini_conf = 0.5$

c: $mini_sup = 0.5, mini_conf = 0.7$

5. Conclusion

Several Researches [4, 9] have shown that the fuzzy association rule can be effectively applied to analyze network traffic, further adopted to the design of Network Intrusion Detection System (NIDS). However, none of them can really achieve their goals in real time because they all used static mining. In other words, they collected a lot of records of packet information first, and then analyze these records by static mining. The main contribute of this paper is to propose an incremental mining algorithm for

generating fuzzy association rule. Thus, as one new record of packet information is come in, the latest set of rules can be generated.

By the definition of fuzzy association rules, mining algorithm for incremental database has to keep each itemset's current support value. That means all possible itemsets and their temporary support values have to be stored in the main memory to speed up the mining process. According to our experiment, no rule will be generated from large itemset with length greater than 6. So the longest itemset length, $maxlen$, set to 6 is enough. Using a letter matrix to represent the product of itemsets and fuzzy sets, our algorithm is successful to complete once mining in several milliseconds to seconds, if the number of features does not excess 21. In the implementation, a sparse matrix is replaced by a string to reduce memory consumption. Thus, all possible itemsets can be stored in memory to speed up mining.

Acknowledgement: Thanks for the support from National Science Council under contract NSC 95-2221-E-130-003.

References

- [1] R. Agrawal, T. Imielinske, and A. Swami, "Mining Association Rules Between Sets of Items in Large Databases," *Proc. of the ACM Intl Conf on Management of data (SIGMOD)*, Vol.22, Issue 2, pp. 207-206, 1993.
- [2] Chin-Chen Chang, Yu-Chiang Li, and Jung-San Lee, "An Efficient Algorithm for Incremental Mining of Association Rules", *RIDE-SDMA*, 2005.
- [3] Walid G. Aref, Mohamed G. Elfeky, and

- Ahmed K. Elmagarmid, "Incremental, Online, and Merge Mining of Partial Periodic Patterns in Time-Series Databases," *IEEE Trans on Knowledge and Data Engineering*, Vol. 16, No. 3, pp. 332-341, 2004.
- [4] Aly El-Semary, Janica Edmonds, Jes'us Gonz'alez-Pino and Mauricio Papa, "Applying Data Mining of Fuzzy Association Rules to Network Intrusion Detection," *Proc. of the IEEE Intl Workshop on Information Assurance United States Military Academy*, pp. 100-107, 2006.
- [5] Dong (Walter) Xie, "Fuzzy Association Rules discovered on Effective Reduced Database Algorithm," *Proceedings of the IEEE Intl Conf on Fuzzy Systems*, pp. 779-784, 2005.
- [6] C. Kuok, A. Fu, and M. Wong, "Mining Fuzzy Association Rules in Databases," *Proc. of the ACM sixth Conf on Information and Knowledge Management*, pp. 209-215, 1997.
- [7] Charu C. Aggarwalm and Philip S. Yu, "A New Approach to Online Generation of Association Rules," *IEEE Trans on Knowledge and Data Engineering*, Vol. 13, Issue 4, pp. 527-540, 2001.
- [8] Hua-Fu Li, Suh-Yin Lee and Man-Kwan Shan, "Online Mining (Recently) Maximal Frequent Itemsets over Data Streams," *Proc. of the IEEE 15th Intl Workshop on Research Issues in Data Engineering: Stream Data Mining and Applications (RIDE-SDMA)*, pp. 11-18, 2005.
- [9] Susan M. Bridges, Rayford B. Vaughn, "Intrusion Detection via Fuzzy Data Mining," *Proc. of Canadian Information Technology Security Symposium*, 2000.
- [10] R. Srikant and R. Agrawal, "Mining Quantitative Association Rules in Large Relational Tables," *Proc. of the ACM Conf. on Management of Data*, pp. 1-12, 1996.
- [11] Wai-Ho Au and Keith C. C. Chan, "Mining Fuzzy Association Rules in a Bank-Account Database," *IEEE Trans on Fuzzy Systems*, Vol. 11, No. 2, pp. 238-248, 2003.
- [12] Pei-Qiliu, Zeng-Zhi Li, and Yin-Liang Zhao, "Algorithm of Mining Fuzzy Association Rules in Network Management," *Proc. of the IEEE Second Intl Conf on Machine Learning and Cybernetics*, pp. 123-127, 2003.
- [13] Ming-Yang Su, "A Fast Association Rules Generating Algorithm by Incremental Mining," *Technical Report*, Ming Chuan University, Oct. 2006.